

PakCommerce AI

Project Proposal



Session: 2023 – 2027

Submitted by:

Talha Rana 2023-CS-605

Faizan 2023-CS-622

Amina Batool 2023-CS-624

Monis Hussain 2023-CS-639

Supervised by:

Ma'am Anam Iftikhar

Department of Computer Science, New Campus
University of Engineering and Technology
Lahore, Pakistan

Contents

List of Figures	ii
List of Tables	iii
1 Proposal Synopsis	1
1.1 Abstract	1
1.2 Introduction	2
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Related Work	3
1.6 Proposed Methodology/System	5
1.7 Tools and Technologies	9
1.8 Team Members Individual Tasks/Work Division	10
1.9 Data Gathering Approach	11
1.10 Timeline/Gantt chart	11
 References	 14

List of Figures

1.1	Tentative Gantt Chart	5
1.2	Tentative Gantt Chart	13

List of Tables

1.1	Related System Analysis	4
1.2	Main Tools and Technologies	10
1.3	Work Division	10
1.4	Tentative Timeline	12

Chapter 1

Proposal Synopsis

1.1 Abstract

PakCommerce AI is an AI-assisted e-commerce platform designed for Pakistani small and medium businesses (SMEs). Many sellers in Pakistan operate across multiple sales channels including Shopify, WooCommerce, WhatsApp, Instagram, Facebook pages, and physical shops. However, their product stock, orders, courier status, and customer messages are typically managed through separate systems, leading to stock mismatches, overselling, delayed order handling, repetitive customer replies, and difficulties in tracking Cash-on-Delivery (COD) orders.

The proposed system provides a unified dashboard where sellers can manage products, inventory, orders, customer conversations, courier information, and business analytics. The project incorporates two AI components: a WhatsApp Sales Agent that handles customer inquiries, checks product availability, collects order details, and creates draft orders; and a seller-facing Business Copilot that answers business questions regarding sales trends, inventory status, COD risk assessment, courier performance, and provides explanations for sales fluctuations. The system also supports selective inventory synchronization, allowing sellers to sync specific products or SKUs across multiple Shopify or WooCommerce stores, including permission-based partner store sharing.

The COD return risk prediction is powered by a machine learning model (Random Forest) trained on Pakistan's largest e-commerce dataset and continuously updated with merchant data, providing explainable risk scores. The system is designed with a flexible architecture, logging all AI actions and requiring human approval for sensitive operations. The ultimate goal is to streamline daily

e-commerce operations for Pakistani sellers while demonstrating practical AI applications in real business environments.

1.2 Introduction

E-commerce is growing in Pakistan, but small sellers still do a lot of work manually. A seller may receive orders from an online store, reply to customers on WhatsApp, update Instagram or Facebook posts, check stock in a spreadsheet and book parcels on courier portals. When all these systems are separate, the seller can easily make mistakes.

For example, one product can be sold from WhatsApp and Shopify at the same time, but the stock may not update on both sides. A customer may ask about size or color on WhatsApp, and the seller has to check inventory manually. After the order is confirmed, the seller may still need to book courier, track delivery and later check COD payment. These steps take time and reduce customer satisfaction.

The project proposes a single platform for Pakistani SMEs. It will focus on product and inventory synchronization, order management, WhatsApp based customer conversations, courier decision support, COD/risk analysis and a natural language copilot for the seller. The project is not a Shopify clone, ERP clone or courier company. It is a smart operations layer that connects with selected existing systems.

The project will use official APIs where possible. WooCommerce provides a REST API for creating, reading, updating and deleting store data[15]. Shopify provides APIs and webhooks for store events such as orders and product updates[10]. WhatsApp communication will be handled through an official provider such as Twilio or Meta Cloud API. Twilio documents that when a customer sends a WhatsApp message, a 24-hour customer service window is opened for free-form replies[13].

1.3 Problem Statement

Pakistani online sellers often manage products, orders, inventory, WhatsApp messages, courier booking and COD payments from different dashboards. Because of this, they face stock mismatch, overselling, slow response time, repeated manual work and difficulty in checking delivery or payment status.

The problem is bigger for SMEs because they usually do not have a large team. The owner or a small staff has to answer customers, update products, confirm orders, book couriers and check payments. They also do not get clear answers to questions like which products are low in stock, which COD orders are risky, which

courier is performing better, and how many orders were handled by WhatsApp.

Therefore, there is a need for a Pakistan-focused e-commerce system that can synchronize selected sales channels, handle WhatsApp customer conversations, create orders, assist the seller through AI and provide simple but useful business insights. There is also a need for selective product-level sync because many sellers manage more than one store and do not always want their full catalog to be synchronized everywhere.

1.4 Objectives

- To build a central seller dashboard with multi-channel synchronization (Shopify/WooCommerce), selective inventory sync, order management, and courier decision support for Pakistani e-commerce sellers.
- To develop an inbound WhatsApp AI Sales Agent using LangGraph that can answer customer questions, check product availability, and create draft orders with human-in-the-loop approval for sensitive actions.
- To implement a seller-facing Business Copilot with a machine learning-based COD risk prediction model (Random Forest trained on Pakistani e-commerce data) that provides explainable business insights, risk scores, and courier recommendations.

1.5 Related Work

Several Pakistani systems address different aspects of e-commerce operations, validating the market need. However, existing solutions primarily focus on operational dashboards or courier management, with limited integration of AI agents for customer interaction and seller assistance. Our project distinguishes itself by combining AI-driven customer engagement, seller copilot capabilities, and explainable COD risk prediction in a single platform.

Research Gap: Existing systems mostly focus on dashboards, courier booking, order management, or COD confirmation. Our proposed project uniquely combines multi-channel synchronization, inbound WhatsApp sales automation, a seller-facing AI copilot, courier decision support, explainable COD/risk scoring, human approval workflows, and complete audit logs in one integrated system.

Unique Contributions of Our Project:

1. Integration of both customer-facing and seller-facing AI agents in a single platform tailored for Pakistani SMEs.

2. Explainable COD risk scoring specifically trained on Pakistani e-commerce data using a Random Forest model.
3. Selective multi-store inventory synchronization with permission-based partner store sharing.
4. AI-driven courier recommendation based on city-level performance metrics.
5. Complete audit trail of AI actions with human-in-the-loop approval for critical operations.

TABLE 1.1: Related System Analysis

Related System	Weakness	Proposed Project Solution
ShipKardo	Primarily an operations platform with limited AI capabilities; no WhatsApp-based sales automation.[9]	Adds WhatsApp AI sales conversation, seller copilot, and tool-based AI actions with full audit trails.
Asaan Retail	Provides business features but lacks AI-first assistant for customer interaction and seller decision support.[1]	Focuses on an AI-first assistant that can talk to customers, perform selected actions, and provide business insights.
Ginkgo Retail	Larger business system with BI reporting; not specifically tailored for Pakistani SME needs.[4]	Targets SMEs with a simpler AI sales agent, explainable copilot, and COD risk scoring based on local data.
ConfirmFlow	Focuses on COD confirmation only; limited to WhatsApp reply parsing for order confirmation. [3]	Extends WhatsApp capabilities to full product Q&A, inventory checking, and draft order placement.
Orderrix	Provides automation for COD sellers but limited to WhatsApp confirmations and address fixing.[6]	Adds multi-channel sync, seller copilot, courier decision support, and comprehensive AI action logging.
Run Courier	Courier-focused with OMS, WMS, and COD settlement; lacks store synchronization.[8]	Combines courier intelligence with store sync, WhatsApp sales automation, and AI-driven insights.

1.6 Proposed Methodology/System

The project will follow an iterative development methodology. We will begin by gathering requirements from sellers and studying available APIs. Subsequently, we will design the database schema, user interface, and integration layer. The system will then be built module by module, with each module tested using sample data and live APIs where feasible.

High Level Architecture

The system is built on a unified backend architecture ensuring consistency and maintainability. It consists of two distinct AI-driven flows:

- 1. Customer-Facing Flow (LangGraph):** WhatsApp conversations are asynchronous and long-running, requiring a system that maintains conversation state. LangGraph provides stateful workflow management, conversation memory, and built-in support for human handoff when the AI is uncertain.
- 2. Seller-Facing Copilot (Vercel AI SDK):** Designed for real-time interaction, the seller dashboard requires low-latency feedback. The Vercel AI SDK is selected for its superior support for streaming text responses, significantly enhancing the perceived performance of the UI for the seller.

Integration Strategy: Both flows are powered by a shared backend infrastructure. By centralizing business logic—including authentication, database interactions, and API wrappers—we eliminate code duplication, minimize bugs, and ensure updates to the core engine reflect across both customer and seller interfaces simultaneously.

All AI actions are logged, and risky actions require seller approval. **Main System**

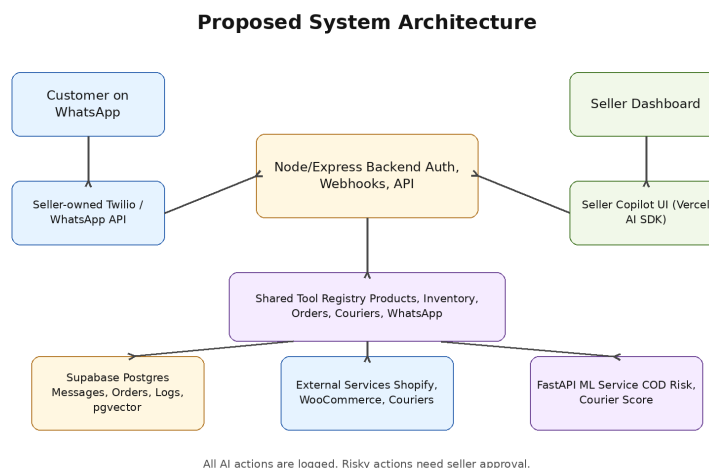


FIGURE 1.1: Tentative Gantt Chart

Modules:

- **Seller Dashboard:** A web-based dashboard where sellers can view products, orders, customers, WhatsApp conversations, courier settings, and analytics.
- **Store Integration Layer:** Connects with Shopify or WooCommerce using seller-provided credentials and APIs.
- **Synchronization Engine:** Imports orders, updates stock, maintains sync logs, supports manual sync triggers, and detects conflicts between connected stores. Supports selective multi-store inventory synchronization.
- **Selective Multi-Store Inventory Sync:** Allows sellers to synchronize only selected products/SKUs across multiple Shopify or WooCommerce stores, with permission-based partner store sharing.
- **WhatsApp AI Sales Agent:** Receives inbound WhatsApp messages, understands customer intent, checks inventory, and creates draft orders.
- **Seller AI Copilot:** Enables sellers to ask natural language questions about low-stock products, high-risk orders, daily sales summaries, and reasons behind sales changes.
- **AI Business Insights:** Generates explainable insights from order, inventory, return, city, and conversation data.
- **RAG Knowledge Base:** Stores product data, FAQs, delivery rules, and policies in pgvector for AI retrieval using actual store data.
- **COD/Risk Scoring:** Provides risk scores using customer order history, previous returns, city-level return trends, order amount, and COD value, with explanations of contributing factors.
- **Courier Decision Support:** Recommends couriers based on delivery success rate, average delivery time, RTO rate, and COD settlement performance.
- **Smart Courier Selection Engine:** Calculates courier scores for each city using weighted metrics.
- **Human Handoff and Approvals:** Routes unclear conversations or sensitive actions to sellers for review.

- **Audit Logs:** Stores all important actions, tool calls, and approvals.

COD Return Risk Prediction Model

The COD return risk prediction will use a supervised machine learning approach:

Dataset:

- **Primary:** "Pakistan's Largest E-commerce Dataset" from Kaggle (<https://www.kaggle.com/datasets/zusmani/pakistans-largest-e-commerce-dataset>) containing order-level data from Pakistani e-commerce transactions.
- **Secondary:** Custom merchant data collected from collaborating Pakistani e-commerce sellers (Shopify/WooCommerce stores).
- **Augmentation:** As new sellers onboard, their order data will be anonymized and added to the training pipeline (with consent) for continuous improvement.

Model Selection:

- **Baseline Model:** Random Forest Classifier - chosen because it works well with tabular data, handles non-linear feature relationships, provides feature importance scores for explainability, is robust to outliers and missing values, and has low risk of overfitting with proper hyperparameter tuning.
- **Evaluation Metrics:** Accuracy, Precision, Recall, F1-Score, AUC-ROC, cross-validation (k-fold), and confusion matrix analysis.

Features Considered:

- Order amount and COD value
- Customer's previous order history (count, return rate)
- City-level return trends
- Product category return rates
- Courier partner historical performance
- Order source (WhatsApp vs. store vs. manual)

Model Lifecycle:

1. Initial training on Kaggle dataset
2. Validation on merchant data (if available)
3. Cold-start handling for new merchants with no history
4. Continuous retraining pipeline every N orders or monthly intervals
5. A/B testing capability for model version comparison

WhatsApp Sales Agent Flow:

1. Customer clicks a Facebook/Instagram ad or WhatsApp link and sends a message.
2. Twilio sends the incoming message to our backend webhook.
3. The system identifies the seller from the WhatsApp number or Twilio messaging service.
4. The AI agent analyzes the customer message and determines intent.
5. The agent uses tools such as searchProducts, checkInventory, and createDraftOrder.
6. If information is missing, the agent asks the customer for size, color, city, address, or phone number.
7. If the customer confirms, a draft order is created and displayed in the seller dashboard.
8. If the agent is uncertain, the conversation is handed over to a human seller.

Seller Onboarding Flow:

1. Seller creates an account on PakCommerce AI.
2. Seller connects Shopify or WooCommerce by entering API credentials.
3. Seller connects WhatsApp using their own Twilio account and WhatsApp number.

4. Seller enters Twilio Account SID, API Key SID, API Key Secret, and Messaging Service SID.
5. The system verifies the connection by receiving a test WhatsApp message.
6. Seller configures rules such as language, delivery charges, handoff rules, and order confirmation policy.
7. The WhatsApp agent becomes active for inbound customer conversations.

In this model, our service does not provide WhatsApp numbers. The seller owns the Twilio account, number, and billing. Our platform only connects to these APIs and runs the AI agent.

1.7 Tools and Technologies

The following stack is tentative. Versions may be updated during implementation, but the system will be designed so that major providers can be changed if needed.

Provider flexibility: The AI model provider will be kept replaceable. The system can use OpenAI, Gemini, Claude, or another provider through an abstraction layer. Similarly, WhatsApp, store, and courier services will be connected through adapters so that the system does not depend on only one provider.

TABLE 1.2: Main Tools and Technologies

Layer	Technology / Tool
Frontend	React or Next.js, TypeScript, Tailwind CSS, shadcn/ui
Backend	Node.js with Express.js
Database	Supabase PostgreSQL
Vector/RAG Storage	pgvector extension in Supabase [11]
AI for WhatsApp Agent	LangGraph for stateful workflow and human handoff [5]
AI for Seller Copilot	Vercel AI SDK UI + AI SDK Core for streaming chat, tools, and structured output [14]
ML Service	Python FastAPI with scikit-learn (Random Forest baseline), pandas, numpy; model registry for version tracking; evaluation metrics (accuracy, precision, recall, F1, AUC-ROC)
Data Pipeline	Apache Airflow or scheduled cron jobs for daily model retraining; DVC for dataset versioning; MLflow for experiment tracking
Messaging	Twilio WhatsApp API initially; Meta Cloud API as alternate/future option
E-commerce APIs	Shopify Admin API and/or WooCommerce REST API
Courier APIs	PostEx[7], TCS[12], BlueEx[2] or manual courier profiles depending on available credentials
Queue/Jobs	Redis and BullMQ for background jobs

1.8 Team Members Individual Tasks/Work Division

TABLE 1.3: Work Division

Team Member Name	Tasks
Amina Batool (2023-CS-624)	Frontend dashboard development, seller screens, copilot UI, approval UI, and analytics pages.
Monis Husain (2023-CS-639)	Backend APIs development, database design, authentication system, sync logs, and store connector implementation.
Talha Rana (2023-CS-605)	WhatsApp AI agent development, LangGraph workflow design, Twilio webhook integration, conversation memory management, and human handoff implementation.
Faizan (2023-CS-622)	RAG pipeline implementation, COD/risk model development (Random Forest implementation, data preprocessing, model training/evaluation), courier decision-support engine..

1.9 Data Gathering Approach

We will collect requirements by distributing questionnaires to Pakistani online sellers who use Shopify, WooCommerce, WhatsApp, and courier services. The questionnaire will be designed to gather information about their daily order processes, inventory management challenges, customer communication patterns, COD issues, courier-related problems, and cases where they manage multiple online stores. We will also study official API documentation and analyze existing local platforms to complement the requirement gathering process.

For training the COD risk prediction model, data will be gathered from multiple sources:

1. **Public Dataset:** Kaggle's "Pakistan's Largest E-commerce Dataset" containing orders, returns, customer demographics, product categories, and city-level data.
2. **Custom Merchant Data:** Anonymized order data from collaborating Pakistani sellers (5-10 merchants initially) including customer order history, COD/return outcomes, courier details, and location data.
3. **Synthetic Data:** Generated to handle class imbalance and test edge cases.
4. **Live Data:** As the system is deployed, orders processed through the platform will continuously feed the training pipeline (with explicit merchant consent and anonymization).

For testing other system modules, we will use sample product catalogs, sample orders with status tracking, anonymized WhatsApp conversations, courier tracking records, and synthetic data where real business data is unavailable. This multi-source approach ensures the model is trained on representative Pakistani e-commerce patterns while maintaining privacy and compliance.

1.10 Timeline/Gantt chart

The following timeline is tentative and can be adjusted after supervisor feedback.

TABLE 1.4: Tentative Timeline

Phase	Duration	Main Work
Phase 1	Weeks 1-2	Requirement gathering through questionnaires, competitor study, and final scope definition.
Phase 2	Weeks 3-4	Architecture design, ERD creation, UI wireframes, and API feasibility check.
Phase 3	Weeks 5-7	Backend setup, database creation, authentication system, and dashboard base implementation.
Phase 4	Weeks 8-10	Shopify/WooCommerce connector development, product/order/inventory sync, selective SKU/store sync rules, and data collection pipeline setup.
Phase 5	Weeks 11-13	WhatsApp AI agent development, Twilio webhook integration, conversation memory management, draft order creation, and begin ML model training (data preprocessing, feature engineering, Random Forest baseline).
Phase 6	Weeks 14-15	Seller copilot development, RAG pipeline implementation, tool calls integration, approval system, complete model training, comparative evaluation of models, and finalize COD risk predictor.
Phase 7	Weeks 16-17	COD/risk scoring integration, AI business insights generation, smart courier selection engine, and audit logs implementation.
Phase 8	Weeks 18-20	Comprehensive testing, bug fixing, final report preparation, presentation creation, and live demo deployment.

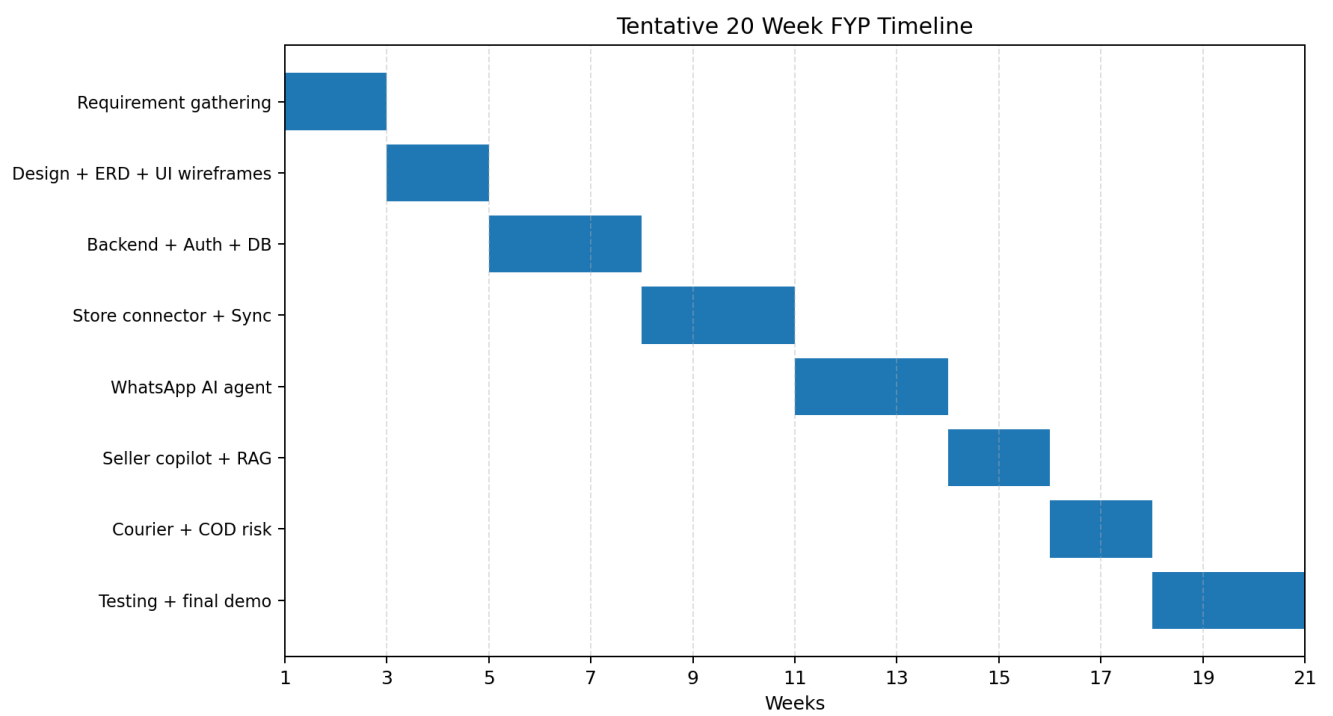


FIGURE 1.2: Tentative Gantt Chart

References

- [1] Asaan Retail. Centralized retail and e-commerce management. <https://asaanretail.io/>. Accessed: 06 June 2026.
- [2] BlueEx. E-commerce logistics and api/plugin integration. <https://www.blue-ex.com/>. Accessed: 06 June 2026.
- [3] ConfirmFlow. Cod operations platform for pakistan. <https://confirmflow.pk/>. Accessed: 06 June 2026.
- [4] Ginkgo Retail. One-stop e-commerce solutions. <https://ginkgoretail.com/>. Accessed: 06 June 2026.
- [5] LangChain. Langgraph overview. <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed: 06 June 2026.
- [6] Orderrix. E-commerce automation for pakistani cod sellers. <https://orderrix.com/>. Accessed: 06 June 2026.
- [7] PostEx. Cod api integration guide. <https://merchant.postex.pk/assets/PostEx%20-%20COD%20API%20Integration%20Guide%20V4.1.6.pdf>. Accessed: 06 June 2026.
- [8] Run Courier. Next generation logistics for pakistan. <https://runcourier.pk.com/>. Accessed: 06 June 2026.
- [9] ShipKardo. Multi-channel ecommerce management software. <https://shipkardo.pk/>. Accessed: 06 June 2026.
- [10] Shopify. Shopify webhooks. <https://shopify.dev/docs/api/webhooks/latest>. Accessed: 06 June 2026.
- [11] Supabase. pgvector: Embeddings and vector similarity. <https://supabase.com/docs/guides/database/extensions/pgvector>. Accessed: 06 June 2026.

-
- [12] TCS. Cod api user manual. <https://envio.tcscourier.com/COD-API-UserManual.pdf>. Accessed: 06 June 2026.
 - [13] Twilio. Key concepts and terms for the whatsapp business platform with twilio. <https://www.twilio.com/docs/whatsapp/key-concepts>. Accessed: 06 June 2026.
 - [14] Vercel. Ai sdk documentation. <https://ai-sdk.dev/docs/introduction>. Accessed: 06 June 2026.
 - [15] WooCommerce. Woocommerce rest api documentation. <https://woocommerce.com/document/woocommerce-rest-api/>. Accessed: 06 June 2026.